STRATOS · Game Theory course project

Four-Member Viva Speaking Plan and Demo

Strategic Behavior in University Course / Project Team Formation
Who says what, what to show, and how to answer: Aryan, Harsh, Ayush, Kashish

Document	STRATOS Viva Speaking Plan and Demonstration Script, version 1.0
Generated	2026-10-26
Built from commit	cdd6ec01fb3d7689d591c7a81152c021a5149992 (branch release/final-product); the PDFs a
Tests recorded on	34e2007: 158 passed, 0 failed, 0 skipped
Demo verified on	real Chrome run of the app files of commit c94f918 (17 states, repo at 7e67155)
Working tree at build	clean
Companion	STRATOS_Complete_Product_Viva_Handbook.pdf
Data	All student data and weights are fictional and synthetic

Contents

How to use this playbook . . . 3

Status you must not overstate . . . 3

1. Speaking order and time allocation . . . 4

2. Speaking cards . . . 5

Card 1 and 5. Kashish: problem, meaning of the results, limits . . . 5

Card 2. Aryan: the model, the equilibrium concept, the engine . . . 8

Card 3. Harsh: the application and the live demonstration . . . 11

Card 4. Ayush: how we know the numbers are right . . . 13

3. Live demonstration runbook . . . 16

3.1 Setup (do this before the viva, and again 60 minutes before) . . . 16

3.2 Path A: the 5-minute demo (Harsh drives) . . . 16

3.3 Path B: the extra segments for the 10-minute demo (run Path A first, then continue here) . . . 17

3.4 If something goes wrong . . . 18

4. Hand-offs and cross-questioning . . . 19

4.1 Transition lines . . . 19

4.2 Questions everyone must be able to answer . . . 19

4.3 If the faculty cross-question across roles . . . 20

5. Final rehearsal checklist (print and tick) . . . 21

Appendix: one-page cheat sheet . . . 22

How to use this playbook

This is the **physical-viva playbook**. The companion document, *STRATOS Complete Product and Technical Viva Handbook*, explains everything in depth; this one tells each of the four of you **what to say, what to show, and what to do**.

1. Everyone reads the handbook (sections 0-6 and 9) first.
2. Each person rehearses their own speaking card aloud, at least twice, with a timer.
3. Run the full demo on the presentation laptop (section 3) the day before and again an hour before.
4. Use the final checklist (section 5) and tick every box.

Four rules that protect every one of you

1. Claim only your own work. Say "I" for what you did. For anything else say "our project does…" or "X built this and I can explain it".

2. Say "stable against unilateral deviations", never "optimal". And say the data is **synthetic**.

3. Do not state anything marked PENDING as done. Pending items are listed below.

4. If you don't know, reason aloud from the model (utility = preference + compatibility − crowding) and say how you would check it.

Status you must not overstate

Item	What you may say	What you must NOT say
Tests	"158 automated tests pass in a fresh environment."	That every possible behaviour is tested.
Independent checking	"An independent recomputation, written from the model document with different code, reproduces every headline number and agrees with the engine."	That a second person's re-implementation or an Excel oracle exists, **unless Ayush has actually produced it** (currently pending).
Browser testing	"We ran the app in a real Chrome browser, and every value on screen matched the engine."	That a person has done a manual walkthrough on the presentation laptop, **until it is done**.
Project title / faculty rules	"Our topic is strategic behaviour in university course / project team formation."	Any official title, group number or marking rule, **until Ayush checks Groups.xlsx and the handout**.
Report and slides	Only what exists on the day.	That a report or slides exist if they do not.

1. Speaking order and time allocation

The story has one thread: **a problem → a model → a working demo → proof it is right → what it means.** Kashish opens and closes, so the audience hears the point twice, in plain words, around the technical middle. Times are **rehearsal targets**, not the faculty's limit; if you are told a different limit, shorten the *extension* blocks first.

#	Speaker	Topic	5-minute version	10-minute version
1	Kashish	Problem, motivation, what we will show (opening)	0:35	1:00
2	Aryan	The model, equilibrium, the engine	1:15	2:45
3	Harsh	The application and the live demonstration	1:45	3:30
4	Ayush	How we know the numbers are right (testing and verification)	0:45	1:30
5	Kashish	What the results mean, limits, future work (closing)	0:40	1:15
	Total		**5:00**	**10:00**

If you are asked to do the 5-minute version, skip every block marked *EXTENSION*. Each card shows both. Questions from the faculty come after the closing; or at any point if they interrupt: answer, then return to the thread with the transition line.

> **Who does what in the demo**
>
> **Harsh drives the mouse and keyboard** and says what each tab shows. When a number or concept needs the maths, Aryan adds one sentence from his seat (cues are in the runbook). Kashish and Ayush do not touch the laptop during the demo; they follow the screen so they can answer questions about it.

2. Speaking cards

Card 1 and 5. Kashish: problem, meaning of the results, limits

> **What Kashish may and may not claim**
> Kashish **presents and explains the finished product**; she does **not** claim to have built, tested or verified it. Say "our project", "the team's engine", "Aryan's model", "Harsh's app". You are expected to understand the whole system and to explain it clearly in plain words.

Opening (5-minute version, about 0:35)

[stand facing the faculty; the app is already open on the Scenario tab]

"Good morning. Our project is called STRATOS. It is about **strategic behaviour in forming university project teams**.

Think about how teams really form. Every student wants a project they like, teammates they work well with, and not a team that is too crowded. But what is good for **you** depends on what **everyone else** picks. That makes it a game.

So we ask two questions. Which assignments are **stable**, meaning nobody can do better by switching on their own? And how do those compare with the best possible assignment? All our student data is fictional, so the numbers illustrate the model. Aryan will explain the model."

[transition: turn and gesture to Aryan]

EXTENSION for the 10-minute version (add about 0:25 after the second paragraph)

"The app has four tabs. **Scenario** shows the students, projects and the numbers behind the game. **Assignment** lets you choose a project for every student and see everyone's utility. **Equilibrium** tells you whether that assignment is stable. **Simulation** lets the students keep switching to their best project and shows where it ends. In plain words: *utility* is how happy a student is; a *Nash equilibrium* is a situation where nobody wants to move alone; *best-response dynamics* is everybody repeatedly taking their best move."

Closing (5-minute version, about 0:40)

[after Ayush; do not touch the laptop]

"So what did we learn? **Stable is not the same as best.** From the same starting assignment, with the same rules, we ended at two different stable assignments. One has total welfare 29, the other 20. Nobody can fix that alone, because fixing it needs several students to move together.

Across all 6,561 possible assignments there are 23 stable ones. The best stable one reaches the maximum welfare of 29; the worst gets only 16. That is a price of anarchy of 1.8125.

The limits: the data is synthetic, we only look at one student moving at a time, and there are no team size limits. With more time we would show these numbers in the app, study groups moving together, and use real data. Thank you; we are happy to take questions."

EXTENSION for the 10-minute version (add about 0:35 before 'The limits')

"Why does the game always settle? Because it has a *potential*: a single number that goes up by exactly the gain of whoever moves. It can't go up forever, so the switching has to stop at a stable assignment. A planner could use this: don't just maximise welfare and don't just let people drift. Look for an assignment that is both high-welfare and stable. In our example one such assignment reaches 29."

What to show

- Opening: the Scenario tab (title, header banner "Real engine detected", and the synthetic-data notice).
- Closing: stay on the Simulation tab with the two results, or the Equilibrium tab on a stable assignment. Point to the numbers; do not read tables.

Likely questions for Kashish, with short answers

[K] Q. In one sentence, what does your project do?

It models students choosing project teams as a game, and finds which assignments are stable and how they compare with the best assignment.

[K] Q. What is a Nash equilibrium, in plain words?

A situation in which nobody can make themselves better off by changing their own choice alone, while everyone else stays put.

[K] Q. Is a stable assignment the best one?

No. Stable means nobody wants to move alone. In our example the best stable assignment has welfare 29 but the worst has 16, and the maximum-welfare assignment is not even always stable.

[K] Q. Why do you use fictional data?

So that we can design the numbers to make strategic behaviour visible in a small example, and because we do not need real student information. We say clearly that the results illustrate the model, not real students.

[K] Q. What does the 'crowding cost' do?

It makes a project less attractive the more students are on it, so that everyone does not simply pile onto their favourite. It grows with the square of the number of teammates.

[K] Q. What is best-response dynamics?

Students take turns switching to whichever project is best for them given what others are doing, until nobody wants to move. Because of the potential, it always stops.

[K] Q. What are the main limitations?

Synthetic data, only single-student moves, no hard team limits, complete information, and exhaustive search that only works for small classes.

Harder follow-ups, and how to reason through them

If they ask…	Reason like this
"Why not just pick the highest-welfare assignment?"	Say what is wrong with it: someone might gain by leaving it. Give the example: the welfare-29 assignment [0,0,1,0,1,1,2,2] is not stable because Eli gains 2 by moving. Then say the other welfare-29 assignment is stable.
"Is welfare the right measure of 'good'?"	It adds up utilities, which assumes they are comparable between students. That is a modelling choice, and a limitation. Other measures (fairness, the worst-off student) are possible.
"Did you build this?"	"I present and explain it; the engine is Aryan's, the app is Harsh's, and the verification work is Ayush's. I can explain every part of it." Do not claim more.
"What would change if students could coordinate?"	Then single-student stability is not enough: groups could move together to a better assignment. That needs coalition concepts (strong equilibrium); we list it as future work.

Must also understand (outside your section)

- The utility equation and what each term means; the numbers 6,561, 23, 29, 16, 1.0 and 1.8125 and where they come from.
- The architecture in one breath: the app calls the engine, the engine knows nothing about the app, tests and saved results sit around both.
- What is verified and what is pending (page 2).

Card 2. Aryan: the model, the equilibrium concept, the engine

What Aryan may claim

The engine, the mathematical model and its documentation (`docs/MODEL.md`, `docs/INTERFACE.md`), the synthetic data, the engine and integration tests, the results generator, and technical sign-off (git history: commits `52f0e6a`, `49645b7`, `859a798`, and the final-release commits). The independent cross-check and several release files were prepared **with AI assistance**; say so if asked.

Opening line

"Thanks, Kashish. I'll explain the model and the engine."

Main points, in order

1. Players, strategies, and the number of assignments: 8 students, 3 projects, $3^8 = 6{,}561$.
2. Utility = preference + compatibility with teammates − crowding cost $\gamma(k-1)^2$. Show the equation on the slide/handbook or write it.
3. Profitable unilateral deviation → Nash equilibrium = stable against unilateral deviations.
4. Potential function: gain = change in Φ → equilibrium exists and dynamics terminates.
5. The engine: separate from the UI, validates input, never mutates, deterministic; checks stability directly rather than trusting the theory.
6. Exhaustive enumeration: all 6,561 assignments → 23 equilibria.

Spoken script, 5-minute version (about 1:15)

"Each of our eight students picks exactly one of three projects. A student's **utility** has three parts: how much they like the project, plus how well they work with the people on the same project, minus a **crowding cost**: gamma times team size minus one, squared. A student alone pays nothing.

If one student can raise their utility by switching alone, that is a **profitable deviation**. An assignment with none is a **pure Nash equilibrium**; we say it is **stable against unilateral deviations**, never 'optimal'.

The reason the maths works is that this game has an **exact potential**: whenever a student moves, one number, the potential Φ, changes by exactly their gain. So an equilibrium always exists, and when students keep switching to their best project, the process has to stop.

The engine is plain Python and completely separate from the interface. It validates its input, never changes it, and checks stability directly. With eight students and three projects there are 6,561 assignments; we check all of them and find 23 equilibria. Harsh will show it running."

[transition: "Harsh, over to you."]

EXTENSION for the 10-minute version (about +1:30; insert after the potential paragraph)

"Let me make it concrete with the two-student example we solved by hand. Student 0 likes project 0 and student 1 likes project 1; together they are compatible, plus two, but a team of two pays a crowding cost of one each. If both choose project 0, their utilities are 6 and 2. Student 1 could move to project 1 and get 5, so that is a profitable deviation, and the assignment is not stable. After the move each is alone and gets 5; nobody gains by moving back, so that is the equilibrium. And the potential went from 7 to 10: up by 3, exactly the gain of the student who moved.

For the default scenario, an equilibrium is found by a pass over every student and every other project: sixteen utility checks. Best-response dynamics visits students in order, or in a random order with a seed, and each student moves only for a strictly positive gain, to the

lowest-numbered best project, so everything is reproducible."

What to show

- The utility and potential equations (handbook sections 3.4 and 3.8), the tiny example table (section 4), or write on a sheet of paper.
- If the faculty want code: `game_engine.py`, functions `utility`, `is_nash`, `best_response_dynamics`; `docs/MODEL.md` for the proof.

Likely questions for Aryan, with short answers

[A] Q. Prove that your game has a pure Nash equilibrium.

Φ takes finitely many values, so some assignment maximises it. From it any unilateral move would raise Φ by the mover's gain, so no move is profitable; hence it is an equilibrium. The key step is that a deviation changes Φ by exactly the deviator's utility change, which holds because compatibility is symmetric and crowding depends only on team size.

[A] Q. Why a quadratic crowding cost?

It is convex, so each extra teammate costs more than the last, it is zero for a lone student, and it depends only on team size, which the potential argument needs. A hard cap would make some deviations infeasible.

[A] Q. What happens if w is not symmetric?

The exact-potential identity can fail, so existence and convergence are no longer guaranteed. The engine rejects asymmetric w when validating.

[A] Q. How do you check Nash in the code?

For each student and each other project it builds the deviated assignment, computes that student's utility, and flags a deviation if the gain is above 1e-9. That is n(m−1) = 16 checks here.

[A] Q. How do you handle ties?

A tie is not a profitable deviation, so a tied student stays; best response is the lowest-index maximiser if the current project is not already best. So equilibria are weak equilibria.

[A] Q. What do 23, 29 and 1.8125 mean?

23 pure equilibria out of 6,561 assignments; maximum total welfare 29; the worst equilibrium has welfare 16, and 29/16 = 1.8125 is the price of anarchy. The best equilibrium reaches 29, so the price of stability is 1.

[A] Q. Is the engine correct?

It is checked by hand-solved fixtures, property tests of the potential and welfare identities, enumeration versus dynamics, and an independent recomputation that agrees exactly (158 tests pass). A second person's re-implementation and an Excel oracle are still to come.

Harder follow-ups, and how to reason through them

If they ask…	Reason like this
"Could best-response dynamics cycle?"	Not here: each move raises Φ by a positive amount, and Φ is bounded, so assignments cannot repeat. With asymmetric w a cycle could occur; that is why we validate.
"Is the equilibrium unique?"	No: there are 23. Which one you reach depends on the start and the update order; we showed 29 and 20 from one start.
"Why is the price of stability 1 but the price of anarchy 1.8?"	Stability asks about the best equilibrium (it reaches the optimum 29); anarchy about the worst (16). Same game, different equilibria.
"Is utility comparable across students?"	It is an assumption. Welfare sums utilities, so it presumes a common scale. Say so; do not defend it as a fact.

Must also understand (outside your section)

- What each tab shows and what a click does (Harsh's part), and the mock-versus-real engine distinction.
- What the tests establish and what is pending (Ayush's part).

Card 3. Harsh: the application and the live demonstration

> **What Harsh may claim**
>
> The Streamlit frontend: `app.py`, `app_helpers.py`, the UI smoke tests, the frontend README draft, and the integration with the engine (git history: `fa0ebbe`, `17cc76a`, `3632842`, `c3d19ff`, `8de4449`, `c94f918`). Including the fix for the deviation count and the loud engine-import error.

Opening line

"Thanks, Aryan. This is the application. Everything you will see comes from Aryan's real engine; the header confirms it."

Main points, in order

1. The app is a thin layer: it stores a project for each student, calls the engine, and draws the results. It never invents its own numbers.
2. Walk the four tabs through one story: Scenario → Assignment → Equilibrium → Simulation.
3. The stable result is always re-verified with a fresh equilibrium check, never trusted from a flag.
4. Show that the same start gives different stable outcomes under different update orders.

Spoken script and actions, 5-minute version (about 1:45)

[sidebar → Scenario preset: choose "Default scenario". Scenario tab is open]

"The header says **Real engine detected**, and the data is marked synthetic. This is the **Scenario** tab: eight students with their skills, three projects, the crowding parameter gamma equals 2, and the preferences and compatibility as heatmaps."

[Assignment tab → in the small seed box type 5 → click "Random"]

"On **Assignment** I can choose a project for each student, or press Random. This random assignment has total welfare **-233** and potential **-60**, and you can see the teams and every student's utility. It's a terrible assignment: almost everybody is crammed onto the same project."

[Equilibrium tab]

"The **Equilibrium** tab says it is **not stable**: 8 students have a profitable move. For example Asha would gain 51.00 by moving to Campus Navigation App."

[Simulation tab; sidebar order = round_robin → click "Run dynamics"]

"On **Simulation**, students switch to their best project in turn. It takes 3 sweeps and 7 moves and ends **Stable**, re-verified with a fresh check. This chart shows welfare climbing to 29."

[sidebar → Student order each sweep: choose "random" → click "Run dynamics" again]

"Now the same start, but students update in a random order. It also ends **Stable**, but after 5 sweeps and 10 moves, and the welfare is only **20**. Two stable outcomes from the same start. Ayush will tell you how we know these numbers are right."

[transition: "Ayush."]

EXTENSION for the 10-minute version (about +1:45; use the Path B steps in the runbook)

- After Path A (steps A1-A6), continue with Path B: **Reset**, then show a *hand-built* assignment: the demo start (Dara, Eli, Farah → Wellbeing Survey Analysis; Goran, Hana → Event Management Portal). Welfare **15**, potential **35**.
- Equilibrium tab: **2 students** have a profitable move (Dara gains 3, Eli gains 4). Point out that the app counts distinct *students*, not deviations.
- Step through the round-robin trace with the slider; Aryan adds that the potential rises 35 → 38 → 44 → 47 → 49.

- Stability versus welfare: set the worst equilibrium [0,2,1,2,2,1,0,0]: welfare **16**, **stable**. Then [0,0,1,0,1,1,2,2]: welfare **29** but **not stable** (Eli gains 2).
- Finish with the **Tiny hand-verified example** preset: from [0,0] one move to [0,1], welfare 8 → 10.

Likely questions for Harsh, with short answers

[H] Q. How does the app get its numbers?

It stores one project index per student in session state, validates it, and calls the engine functions: utilities, teams, welfare, potential, is_nash and best_response_dynamics. The helpers then turn the results into tables and charts.

[H] Q. What if the engine is missing or broken?

If it is missing, the app falls back to a clearly labelled mock. If it is present but fails to import, it shows a red error with the traceback instead of quietly using the mock. Setting STRATOS_REQUIRE_REAL_ENGINE=1 makes any fallback a hard error.

[H] Q. Why does the final status say 'Re-verified'?

The dynamics returns a 'converged' flag, but stability is a separate claim, so the app calls the equilibrium check again on the final assignment. We never trust the flag alone.

[H] Q. What happens with invalid input?

The controls only offer valid choices, and any engine error is shown as a message, not a crash.

[H] Q. What did you fix recently?

The Equilibrium tab used to count deviation entries, so all-on-one-project said '16 students' for 8 students. It now counts distinct students, with a regression test. We also replaced the deprecated width setting and raised the Streamlit minimum to 1.51.

[H] Q. What are the known UI defects?

Two cosmetic ones: an empty rounded bar above the title, and the Simulation chart's legend overlaps its title. Neither affects results.

[H] Q. How is the UI tested?

Streamlit's AppTest runs the app headlessly: no exceptions, four tabs, Run dynamics works, real engine used, the 8-not-16 count. We also drove real Chrome and checked on-screen values; a manual run on the demo laptop is still to do.

Harder follow-ups, and how to reason through them

If they ask…	Reason like this
"Why did the two orders end at different places?"	Each order is a different path; each path stops at some equilibrium. There are 23, so different paths can end in different ones. The potential guarantees it stops, not where.
"Is the mock a risk to the demo?"	With the real engine present it is never used, and the header and screenshots show 'Real engine detected'. A test asserts it, and strict mode removes the risk.
"What does Reset do and why is welfare so negative?"	Reset puts all 8 students on project 0, so every one pays c(8) = 2·49 = 98. Welfare is -670. It shows how strongly the crowding cost bites.

Must also understand (outside your section)

- The utility equation and why stable ≠ optimal. What the potential is and why dynamics stops.
- The headline results (6,561 / 23 / 29 / 16 / 1.8125) and which are verified.

Card 4. Ayush: how we know the numbers are right

IMPORTANT for Ayush: claim only what you did

At the time this document was built, **no commits by Ayush were found** in the repository, and none of the plan's verification deliverables (`tests/oracle/`, Excel oracle, `docs/TEST_LOG.md`, report, slides, `docs/VIVA.md`) are present. That may be work you did elsewhere; this document can only describe what the repository shows.

The script below is therefore written about **the team's evidence**. Wherever it says "we", you may say it as the team. Wherever a **[BRACKET]** appears, say it **only if it is true**, in your own words; otherwise use the alternative.

You should be able to explain every test group and number below; you do not have to have written them.

Opening line

"Thanks, Harsh. A fair question after any demo is: how do we know these numbers are right? Here is how we checked."

Main points, in order

1. Automated tests: **158 passed, 0 failed, 0 skipped**, run in a fresh environment on a clean checkout, with the log saved in `docs/results/pytest_output.txt`.
2. What the groups cover: engine unit and property tests, the engine-to-UI contract, the Streamlit app tests, and the independent cross-check.
3. Independent recomputation: from the model document, with different code and exact arithmetic, it reproduces 6,561 assignments, 23 equilibria, maximum welfare 29, price of stability 1, price of anarchy 29/16, and agrees with the engine.
4. Real-browser check: every value on screen matched the engine in a real Chrome session; screenshots and a log are saved.
5. Honesty about what is still pending.

Spoken script, 5-minute version (about 0:45)

"We checked the results in four ways. First, **158 automated tests pass** in a fresh environment: hand-solved examples, property tests of the potential and welfare identities, tests of the engine-to-app contract, and tests of the app itself. The log, with the exact code version, is saved in the repository.

Second, we **recomputed everything independently** from the model document, with different code and exact fractions. It reproduces 6,561 assignments, 23 equilibria, a best welfare of 29, a price of stability of 1 and a price of anarchy of 29 over 16, and it agrees with the engine on every profile.

Third, we ran the app in a **real browser** and compared every number on screen with the engine. They matched.

What is **still pending**: [a second person's independent re-implementation and Excel check — say this if it is not yet done] and a manual run on this laptop. We would rather say that than overclaim."

[transition: "Kashish will say what it all means."]

EXTENSION for the 10-minute version (about +0:45)

"What these tests cannot do: they check that the **code matches the model**, not that the model matches real students. The data is synthetic, and we say so. One test, I-06, deliberately fails if anyone changes the data without regenerating the saved results, so the evidence can never go stale. We also found and fixed real problems along the way: the plotting library was missing from the install file, which broke a fresh install; and one message counted moves instead of students."

Alternative lines, depending on what is true on the day

If this is true	Say
Ayush has written the independent re-implementation / Excel oracle and it agrees	"I rebuilt it independently from the model document, without looking at the engine, and the equilibrium set and welfare matched." (Show the file and output.)
It is not done	"A second-person re-implementation and an Excel oracle are planned; what we have today is an independent recomputation prepared by the engine's owner, which is independent of the engine's code but not of its author."
Ayush has checked `Groups.xlsx` and the handout	State the official title and rules he found, and where he recorded them.
Not checked	"We are confirming the exact title against the official sheet." Do not guess.

What to show

- `docs/results/pytest_output.txt` and `docs/results/independent_crosscheck_output.txt` (the last line: IDENTICAL).
- `screenshots/INDEX.md`. The terminal: `python -m pytest -q` if time allows (about 20 seconds).

Likely questions for Ayush, with short answers

[Ay] Q. How do you know the engine is right?

Several independent layers: hand-solved examples, property tests of the potential identity on random games, enumeration compared with dynamics, and a separate recomputation that agrees on every headline number.

[Ay] Q. What does 'independent' mean here?

It means different code, from the written model, with exact arithmetic, and it does not import the engine for its own numbers. It does not mean a different author: the person who owns the engine prepared it. A second person's check is still to come.

[Ay] Q. What do the tests not cover?

Whether the model is realistic, visual details of the UI, and anything on a machine we have not run. The browser check was automated, not a manual walkthrough.

[Ay] Q. What is a regression test? Give an example.

A test that pins a bug fix. When the Equilibrium tab showed 16 students for 8, we added a test that asserts the distinct-student count, so it cannot come back.

[Ay] Q. What is the difference between unit, integration and smoke tests here?

Unit tests check engine functions against hand answers; integration tests check the documented contract between engine and app; smoke tests check that the app starts, has four tabs, and a button works.

[Ay] Q. Could the numbers be wrong in both the engine and the cross-check?

In principle, if the model itself were mis-specified, since both follow the same specification. That is why a second person's re-implementation and a hand-computed spreadsheet matter, and why the tiny example was solved by hand first.

[Ay] Q. Do the documents, tests and app agree?

Yes on the headline numbers: they all come from the same saved report. The test count in the log is 158. Any change to data or engine makes the saved-results test fail until the evidence is regenerated.

Harder follow-ups, and how to reason through them

If they ask…	Reason like this
"Did you write the engine tests?"	Answer truthfully. If not: "The engine tests are Aryan's; I reviewed [what you reviewed] and I can explain what each group checks."
"Show me one test and say what it proves."	Pick the tiny-example table: at [0,0] utilities are 6 and 2 because c(2)=1; at [0,1] they are 5 and 5, and the potential rose from 7 to 10 by exactly the mover's gain.
"What if your test suite passes but the app is wrong?"	Passing tests only cover what they check. That is why we also compared the screen with the engine in a real browser, and why a manual run is on our checklist.

Must also understand (outside your section)

- The utility equation, the potential, and why stable ≠ optimal.
- What each tab shows, and how the app uses the engine.

3. Live demonstration runbook

Every instruction below was executed against the real running app in Chrome, and every value on screen was compared with the engine (`screenshots/INDEX.md`; 17 captured states). Labels are copied from the app.

3.1 Setup (do this before the viva, and again 60 minutes before)

```
cd STRATOS
.venv\Scripts\activate
python -m pip install -r requirements.txt     # only the first time
streamlit run app.py                          # opens http://localhost:8501
# optional: refuse any mock fallback
$env:STRATOS_REQUIRE_REAL_ENGINE = "1"; streamlit run app.py
```

- ☐ The page header shows the green box **Real engine detected: game_engine.py and sample_data.py loaded.**
- ☐ Sidebar: **Scenario preset** = *Default scenario*; **Student order each sweep** = *round_robin*; **Dynamics seed** = 7; **Max sweeps** = 50.
- ☐ Browser zoom 100-125%, window maximised, notifications off, laptop charging, Wi-Fi not needed.
- ☐ Second terminal ready: `python game_engine.py --report` and `python scripts/independent_crosscheck.py --compare` (backup evidence).

3.2 Path A: the 5-minute demo (Harsh drives)

#	Do this	What should appear (verified)	Say
A1	Sidebar → **Scenario preset** → *Default scenario*. Stay on the **Scenario** tab.	Eight students with skills, three projects, **Crowding parameter (gamma) 2**, two heatmaps. Header: "Real engine detected" and the synthetic-data note.	"Real engine, synthetic data. Eight students, three projects."
A2	**Assignment** tab → click the small seed box, type **5**, press Enter → click **Random**.	**Social welfare W(s) -233**, **Potential Φ(s) -60**. Teams: most students on one project.	"Total welfare minus 233: everyone is crammed together."
A3	**Equilibrium** tab.	Yellow box: **not stable — 8 student(s) have a profitable unilateral move**. Example: Asha gains 51.00. A table of deviations.	"Not stable: every student has a better move."
A4	**Simulation** tab (order = round_robin) → **Run dynamics**.	**Sweeps run 3**, **Moves made 7**, **Status Stable**, green "Re-verified: the final assignment is a pure Nash equilibrium." Teams: C: Asha, Bram, Dara; W: Chen, Farah; E: Eli, Goran, Hana. Chart: welfare rises to 29.	"Settles in 3 sweeps; the final check is fresh, not a flag."
A5	Sidebar → **Student order each sweep** → *random* → on the Simulation tab click **Run dynamics** again.	**Sweeps run 5**, **Moves made 10**, **Status Stable**. Teams: C: Bram, Eli, Hana; W: Asha, Chen, Farah; E: Dara, Goran. Welfare ends at 20.	"Same start, different order: also stable, but welfare 20 instead of 29."
A6	(Aryan, from his seat)	—	"Both are Nash equilibria. Stable does not mean best: of 6,561 assignments, 23 are stable, with welfare from 16 to 29."

3.3 Path B: the extra segments for the 10-minute demo (run Path A first, then continue here)

#	Do this	What should appear (verified)	Say
B1	After A6: **Assignment** tab → **Reset**.	Everyone on Campus Navigation App: welfare **-670**, potential **-199**.	"Everyone on one project: the crowding cost is huge."
B2	Set the demo start with the selectboxes: **Dara, Eli, Farah** → *Wellbeing Survey Analysis*; **Goran, Hana** → *Event Management Portal*; Asha, Bram, Chen stay on *Campus Navigation App*.	Welfare **15**, potential **35**; utilities Asha 1, Bram 3, Chen −2, Dara −2, Eli −3, Farah 4, Goran 9, Hana 5.	"Eli has the lowest utility, −3."
B3	**Equilibrium** tab.	**not stable — 2 student(s)**: Dara gains 3 (−2 → 1), Eli gains 4 (−3 → 1), both to Event Management Portal.	"Two students can gain by moving alone."
B4	**Simulation** tab; sidebar order → *round_robin* → **Run dynamics**. Move the **Inspect profile after move #** slider.	Sweeps 3, Moves 4, Stable. Trace: Dara W→E (+3); Chen C→W (+6); Dara E→C (+3); Eli W→E (+2). Final teams C: Asha, Bram, Dara; W: Chen, Farah; E: Eli, Goran, Hana.	Aryan: "The potential goes 35, 38, 44, 47, 49: up by exactly each mover's gain."
B5	Sidebar order → *random* (dynamics seed 7) → **Run dynamics**.	Sweeps 4, Moves 4, Stable. Final teams C: Asha, Bram, Hana; W: Chen, Farah; E: Dara, Eli, Goran (welfare 23).	"Different order, different stable assignment."
B6	**Assignment** tab → set the **worst equilibrium** [0,2,1,2,2,1,0,0]: Asha→C, Bram→E, Chen→W, Dara→E, Eli→E, Farah→W, Goran→C, Hana→C. Then **Equilibrium** tab.	Welfare **16**, potential **38**. Equilibrium tab: green **stable against unilateral deviations**.	"Stable, yet the welfare is 16, far below 29."
B7	**Assignment** tab → set [0,0,1,0,1,1,2,2]: Asha, Bram, Dara → *Campus Navigation App*; Chen, Eli, Farah → *Wellbeing Survey Analysis*; Goran, Hana → *Event Management Portal*. Then **Equilibrium** tab.	Welfare **29**, potential 47. **not stable — 1 student(s)**: Eli gains 2 by moving to Event Management Portal.	"The best welfare is not even stable. Optimal and stable are different questions."
B8	Sidebar → order *round_robin*; preset *Tiny hand-verified example*. **Simulation** → **Run dynamics**.	Moves made **1**, Stable; [0,0] → [0,1].	"The two-student example we solved by hand."

Closing numbers (not in the UI: say or show from the terminal)

`python game_engine.py --report` → 6,561 assignments, **23** equilibria, maximum welfare **29**, price of stability **1.0**, price of anarchy **1.8125**. `python scripts/independent_crosscheck.py --compare` → ends with `comparison with game_engine: IDENTICAL` (about 4 seconds).

3.4 If something goes wrong

Problem	Do this	Say
App does not start	Check the venv is active: `.venv\Scripts\activate`; `python -m pip install -r requirements.txt`; try `--server.port 8765`.	"One moment, I'll start it again." If it cannot start in 60 seconds, switch to the screenshots.
Red error box 'failed to import'	Open a terminal: `python game_engine.py --report` shows the real error. Restore the file with `git checkout -- game_engine.py` only if you are sure.	"The app protects us from using a stand-in engine." Move to screenshots.
A chart is blank or slow	Press **R** in the browser to rerun, or refresh the page; the tables still show the same numbers.	"The numbers are in the table; the chart is the same data."
A number differs from this script	Check the sidebar: preset, order, dynamics seed (7), and the assignment seed box. Then compare with `python game_engine.py --report`.	"Let me check which scenario and seed we are on." Never defend a wrong number.
Laptop or browser freezes	Use the saved screenshots in `screenshots/` (see `screenshots/INDEX.md`) and the terminal outputs.	"These are real captures from the running app, taken at commit c94f918."
Someone asks for a click you have not rehearsed	Do it if it is simple (e.g. change gamma in the Custom preset); otherwise say what you expect and why.	"I expect crowding to push teams apart; let me check."

4. Hand-offs and cross-questioning

4.1 Transition lines

From → to	Say
Kashish → Aryan	"That is the idea in plain words. Aryan will now explain the model behind it."
Aryan → Harsh	"Harsh will show it running on the real engine."
Harsh → Ayush	"Two stable outcomes from one start: so how do we know these numbers are right? Ayush."
Ayush → Kashish	"So the numbers are checked. Kashish will say what they mean."
Anyone back to the thread after a question	"To pick up where we were: ..." and name the next step (model, demo, evidence, meaning).

4.2 Questions everyone must be able to answer

[all] Q. What problem does STRATOS address?

Forming project teams when each student's outcome depends on others' choices; we ask which assignments are stable and how they compare with the best.

[all] Q. Write or state the utility.

u = preference for my project + compatibility with the teammates on it − crowding cost $\gamma(k-1)^2$, where k is the team size including me.

[all] Q. What is a Nash equilibrium here?

An assignment in which no single student can raise their own utility by switching alone: stable against unilateral deviations.

[all] Q. What is best-response dynamics, and does it stop?

Students take turns switching to their best project. It always stops, because the potential rises with every move and cannot rise forever.

[all] Q. Is a stable assignment the best?

No. Our 23 stable assignments have welfare from 16 to 29, and the maximum-welfare assignment may not even be stable.

[all] Q. How is the project organised?

A Streamlit app (four tabs) calls a separate Python engine; tests, saved results and an independent cross-check surround both.

[all] Q. What do the main results mean, and what is pending?

6,561 assignments, 23 equilibria, best welfare 29, price of stability 1, price of anarchy 1.8125. Pending: a second person's independent oracle, a manual walkthrough on this laptop, and the official title check.

4.3 If the faculty cross-question across roles

Asked of…	About…	A good short answer
Aryan	the frontend	"Harsh built the app. It is a thin layer: one project index per student goes to the engine; the engine's numbers come back and are drawn. I wrote the contract it follows, `docs/INTERFACE.md`."
Aryan	testing	"There are hand-solved fixtures, property tests of the potential identity, enumeration compared with dynamics, and an independent recomputation. The latest log says 158 passed."
Harsh	the model	"Utility is preference plus compatibility with teammates minus a crowding cost. A student alone pays no crowding. The Equilibrium tab checks whether any student has a better move alone."
Harsh	the potential	"A single number Φ that moves by exactly a mover's gain. So switching always stops. Aryan proved it in docs/MODEL.md; I can walk through the steps."
Ayush	the engine	"The engine checks stability by trying every student's every other project and comparing utilities; it validates input and does not change it. It does not rely on the theory to claim stability."
Ayush	the frontend	"Four tabs: Scenario, Assignment, Equilibrium, Simulation. The app re-checks the final state with a fresh equilibrium test. I verified [what you actually checked]."
Kashish	the architecture	"The app talks to the engine through a documented contract. The engine does not know about the app. Tests and a results script sit around them, and the app says whether it is using the real engine."
Kashish	the equations	"Utility = preference + teammate compatibility − crowding cost. Social welfare is everyone's utility added up. The potential is a number that changes by exactly the mover's gain."

5. Final rehearsal checklist (print and tick)

Reading and rehearsal

- ☐ All four members have read the **Handbook** sections 0-6 and 9.
- ☐ Each member has rehearsed their **speaking card** aloud at least twice with a timer (5-minute and 10-minute versions).
- ☐ Everyone can answer the **seven questions everyone must answer** (section 4.2) without notes.
- ☐ Everyone has read the **ownership map** (Handbook section 10) and knows what they may claim.
- ☐ One full **dress rehearsal** of both versions, with a second person asking cross-questions.

Technical readiness on the presentation machine

- ☐ Repository cloned or copied; `.venv` created; `python -m pip install -r requirements.txt` finished with no errors.
- ☐ `python -m pytest -q` shows **158 passed** (about 20 seconds). Write the number you saw: .....
- ☐ `streamlit run app.py` opens; the header shows **Real engine detected**; no red error box.
- ☐ Path A rehearsed end to end; Path B rehearsed; the on-screen numbers match the runbook.
- ☐ `python game_engine.py --report` and `python scripts/independent_crosscheck.py --compare` both run (backup evidence).
- ☐ A **manual walkthrough** of all four tabs and both update orders has been done on this machine by a team member (Harsh): date / initials: .....
- ☐ Backup: `screenshots/` folder and the two PDFs are on the laptop and on a USB stick.

Evidence consistency

- ☐ `git rev-parse --short HEAD` on the presentation machine = the commit on the PDFs' cover (**cdd6ec0**) or a descendant that only changes documents. If not, rerun the tests.
- ☐ `docs/results/pytest_output.txt` shows the same test count as the slide/report you will quote.
- ☐ The numbers in the report and slides (if they exist) match: 6,561 / 23 / 29 / 16 / 1.0 / 1.8125.
- ☐ **Official title and group verified** against `Groups.xlsx` and the handout (Ayush): title recorded: .....
- ☐ README, report, slides and screenshots exist and are consistent (tick only those that exist; leave the rest unticked and do not mention them as done).

On the day

- ☐ Laptop, charger, HDMI/USB-C adapter, mouse, pointer; printed copy of both PDFs for annotation; pen.
- ☐ Phone on silent; browser notifications off; battery saver off; screen timeout off.
- ☐ Everyone arrives early; agree who answers if the question is about the whole project (default: whoever owns the topic, then add).
- ☐ Final check: **no member claims work they did not do.** If unsure, say "I understand it and can explain it, but X built it."

Appendix: one-page cheat sheet

Quantity	Value
Utility	$u_i(s) = p(i, s_i) + \Sigma\, w(i,j)$ over teammates $- \gamma(k_i - 1)^2$; $\gamma = 2$ by default
Social welfare / potential	$W = \Sigma\, u_i$; $\Phi = \Sigma\, p + \Sigma_{\text{same-team pairs}}\, w - \Sigma_t\, \Sigma_{k=1..n(t)}\, c(k)$; a move changes Φ by exactly the mover's gain
Assignments / equilibria	$3^8 = 6{,}561$; 23 pure Nash equilibria; 20 of them Pareto-optimal
Maximum welfare	29 (two assignments; [0,0,1,0,2,1,2,2] is stable, [0,0,1,0,1,1,2,2] is not: Eli gains 2)
Worst equilibrium	welfare 16; price of stability 1.0; price of anarchy 1.8125 = 29/16
Demo start [0,0,0,1,1,1,2,2]	welfare 15, Φ 35; round-robin → welfare 29 in 3 sweeps / 4 moves; random (seed 7) → welfare 23 in 4 sweeps / 4 moves
Path A start (Random, seed 5)	[2,1,2,1,2,2,2,2], welfare -233, Φ -60; round-robin → 29 (3 sweeps, 7 moves); random (seed 7) → 20 (5 sweeps, 10 moves)
Tiny example	p = [[5,1],[1,5]], w = 2, γ = 1: [0,0] → u = (6,2), W 8, Φ 7, not stable; [0,1] → u = (5,5), W 10, Φ 10, stable
Tests / evidence	158 passed, 0 failed, 0 skipped (commit 34e2007); independent cross-check IDENTICAL
Say / never say	"stable against unilateral deviations", "synthetic data" / never "optimal", never claim pending work